

Tarea 6

Gestión de energía y modos de ahorro en el ESP32

SOLUCIÓN

Sistema con tarea activa, señalización mediante LED RGB, entrada automática a deep sleep y despertar por temporizador RTC o pulsador externo EXT0.

Asignatura	Sistemas Embebidos
Framework	Arduino sobre PlatformIO
Microcontrolador	ESP32 DevKit v1
Simulación	Wokwi Online y Wokwi para VS Code
Repositorio	github.com/JdesaG/tarea6-esp32-gestion-energia
Fecha	07 de agosto de 2026

Autor / estudiante: JdesaG

Documento de implementación, análisis y evidencias

1. Descripción del ejercicio

Se desarrolló un sistema embebido que alterna de forma cíclica entre actividad y bajo consumo. Tras cada arranque o despertar, el ESP32 informa la causa por el puerto serial, indica su estado con un LED RGB, ejecuta una tarea activa de cinco segundos y configura dos fuentes de despertar antes de entrar en deep sleep.

La primera fuente que ocurra despierta al microcontrolador: el temporizador RTC después de 10 segundos o un nivel bajo en GPIO33 producido por el pulsador. El despertar desde deep sleep provoca un nuevo arranque desde `setup()`; por ello los contadores se almacenan en memoria RTC con `RTC_DATA_ATTR`.

Objetivo de diseño

- Reducir el tiempo de CPU activa cuando el sistema no tiene una tarea útil que ejecutar.
- Combinar un despertar periódico y un evento externo en una implementación verificable.
- Separar la lógica de energía, la señalización y el flujo principal en módulos claros.

Cumplimiento de requisitos

Requisito	Implementación	Estado
Tarea activa temporizada	Cinco iteraciones de 1 s con telemetría serial.	Cumple
Modo de ahorro explícito	Deep sleep mediante <code>esp_deep_sleep_start()</code> .	Cumple
Despertar por temporizador	<code>esp_sleep_enable_timer_wakeup(10 s)</code> .	Cumple
Despertar externo	<code>esp_sleep_enable_ext0_wakeup(GPIO33, LOW)</code> .	Cumple
Estados visibles	LED RGB y mensajes seriales estructurados.	Cumple
Código estructurado	Módulos <code>power_manager</code> , <code>status_led</code> y <code>main</code> .	Cumple
Simulación Wokwi	Circuito publicado y archivos VS Code incluidos.	Cumple

DECISIÓN

Se eligió deep sleep porque apaga CPU y periféricos digitales, permite conservar datos RTC y admite el temporizador RTC y EXT0 solicitados.

2. Circuito en Wokwi

El circuito fue creado y publicado en Wokwi Online. La siguiente imagen es la captura generada por Wokwi para el proyecto público y muestra el cableado almacenado en `diagram.json`.

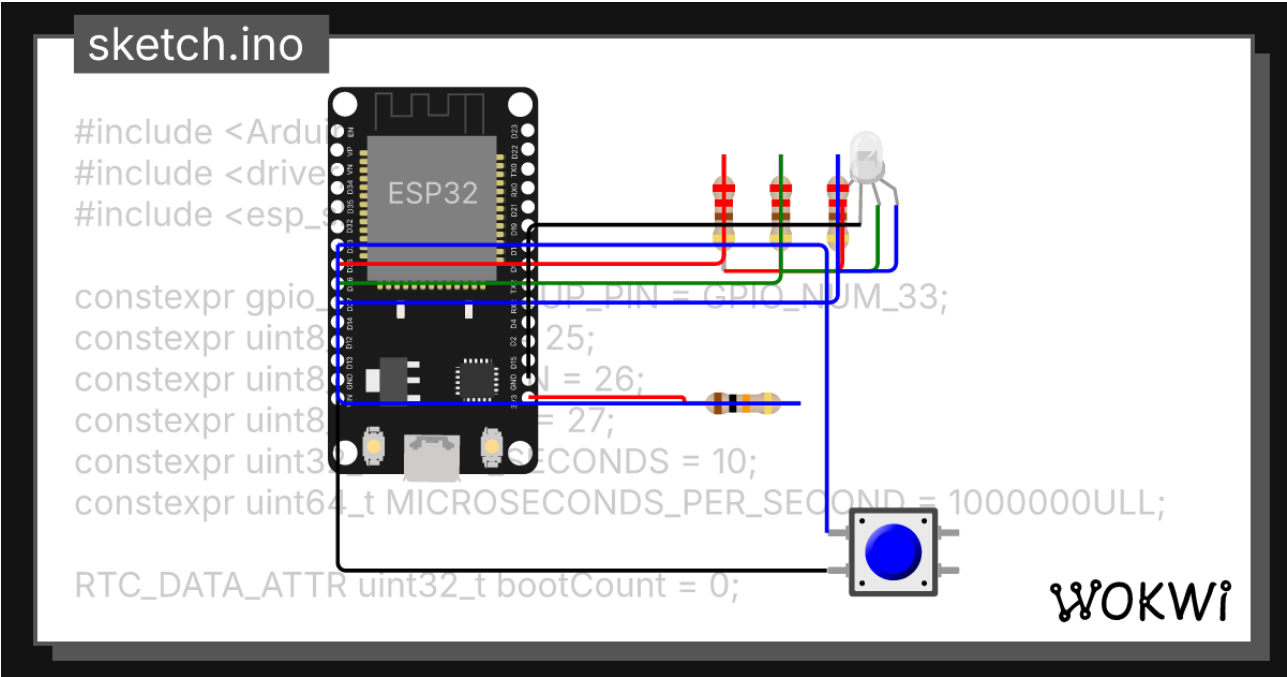


Figura 1. ESP32, LED RGB, resistencias limitadoras y pulsador EXT0 en Wokwi.

Enlace directo

<https://wokwi.com/projects/471748304593999873>

Conexiones

Señal	GPIO / conexión	Función
Rojo	GPIO25 -> 220 ohm -> R	Preparación para reposo
Verde	GPIO26 -> 220 ohm -> G	Tarea activa
Azul	GPIO27 -> 220 ohm -> B	Arranque o despertar
Común RGB	COM -> GND	LED de cátodo común
Wake EXT0	GPIO33 -> pulsador -> GND	Despertar al nivel LOW
Pull-up	3.3 V -> 10 kohm -> GPIO33	Evita entrada flotante

INTERACCIÓN

Durante el reposo, se puede hacer clic en el pulsador azul o mantener la tecla B. Si no se pulsa, el temporizador despierta al ESP32 automáticamente.

3. Funcionamiento del sistema

Secuencia de estados

Orden	Estado	LED	Duración / salida
1	DESPIERTO	Azul	1 s; causa y contadores por serial
2	ACTIVO	Verde intermitente	5 s; TAREA_ACTIVA=1/5 ... 5/5
3	PREPARANDO_SLEEP	Amarillo	1 s; configura TIMER y EXT0
4	SLEEP	Apagado	Hasta 10 s o pulsación externa
5	NUEVO ARRANQUE	Azul	Vuelve a setup() y registra la causa

Causas de despertar

- Temporizador: el controlador RTC genera el evento después de 10 segundos de reposo.
- EXT0: GPIO33 es un GPIO RTC. El pulsador lo lleva a LOW y despierta el chip.
- Primer arranque: la causa es indefinida porque todavía no se ha producido deep sleep.

Tratamiento del pulsador

Antes de dormir, el firmware espera que el pulsador esté liberado. Esto evita entrar al modo de reposo con la condición EXT0 ya activa y despertar inmediatamente. El pull-up externo de 10 kohm y el pull-up RTC mantienen GPIO33 estable durante el reposo.

Persistencia RTC

Se mantienen tres variables: número total de arranques, despertares por temporizador y despertares por EXT0. Esta información permite verificar el comportamiento después de cada reinicio provocado por deep sleep.

```
RTC_DATA_ATTR uint32_t bootCount = 0;
RTC_DATA_ATTR uint32_t timerWakeCount = 0;
RTC_DATA_ATTR uint32_t externalWakeCount = 0;

esp_sleep_enable_timer_wakeup(10ULL * 1000000ULL);
esp_sleep_enable_ext0_wakeup(GPIO_NUM_33, LOW);
esp_deep_sleep_start();
```

4. Código y estructura

Archivo	Responsabilidad
src/main.cpp	Secuencia principal, tarea activa, reporte y contadores RTC.
src/power_manager.cpp	Causas, validación de errores, fuentes de despertar y deep sleep.
src/status_led.cpp	Abstracción de los cuatro estados visuales del LED RGB.
diagram.json	Partes, posiciones, atributos y conexiones del circuito Wokwi.
wokwi.toml	Rutas al firmware BIN y ELF generado por PlatformIO.
wokwi-web/sketch.ino	Variante monolítica equivalente publicada en Wokwi Online.
tests/*.yaml	Escenarios para TIMER y pulsador EXT0 en Wokwi CI.
tests/validate_project.py	20 comprobaciones de coherencia sin servicios externos.

Fragmento principal

```
void setup() {  
    const auto cause = esp_sleep_get_wakeup_cause();  
    registerWakeup(cause);  
    printStartupReport(cause);  
  
    StatusLed::blue();  
    runActiveTask();  
    prepareForSleep();  
    PowerManager::enterDeepSleep();  
}
```

Mensajes seriales

Los mensajes tienen claves estables (BOOT, CAUSA, ESTADO y CONTADORES) para que una persona o un escenario automatizado pueda verificar el flujo sin depender únicamente del LED.

Repositorio

<https://github.com/JdesaG/tarea6-esp32-gestion-energia>

REPRODUCIBLE

El repositorio contiene código fuente, README, instrucciones, circuito Wokwi, configuración PlatformIO y escenarios de prueba.

5. Pruebas y evidencias

Prueba	Resultado	Evidencia
Compilación PlatformIO modular	APROBADA	RAM 6.9%; flash 21.5%
Compilación variante Wokwi web	APROBADA	Firmware BIN generado
Validez JSON	APROBADA	diagram.json parseado
Linter Wokwi CLI 0.26.1	APROBADA	0 errores; 1 nota informativa
Coherencia firmware-circuito	APROBADA	20/20 comprobaciones
Publicación Wokwi	APROBADA	HTTP 200 y captura pública
Publicación GitHub	APROBADA	HTTP 200 y rama main

Salida de compilación principal

```
No dependencies
Building in release mode
Retrieving maximum program size .pio/build/esp32dev/firmware.elf
Checking size .pio/build/esp32dev/firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM:  [=          ]  6.9% (used 22712 bytes from 327680 bytes)
Flash: [==         ] 21.5% (used 281561 bytes from 1310720 bytes)
===== [SUCCESS] Took 1.02 seconds =====
```

Salida de compilación de la variante web

```
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM:  [=          ]  6.9% (used 22712 bytes from 327680 bytes)
Flash: [==         ] 21.5% (used 281205 bytes from 1310720 bytes)
Building .pio/build/esp32dev/firmware.bin
esptool.py v4.9.0
Creating esp32 image...
Merged 3 ELF sections
Successfully created esp32 image.
===== [SUCCESS] Took 5.52 seconds =====
```

Integridad del firmware

```
4143f360a99bac921b172b4172b6a4b901d17b5583b5ea50909e68c47acf77dd  .pio/build/esp32dev/firmware.bin
82f65dc6ace4b19f90da8e857af35c4d2fc0970707223e79907aeb279fdf5371  .pio/build/esp32dev/firmware.elf
```

Los hashes SHA-256 permiten confirmar que los binarios evaluados son exactamente los generados durante esta entrega.

6. Procedimientos de simulación

Prueba A - Despertar por temporizador

- Abrir el enlace Wokwi, iniciar la simulación y no pulsar el botón.
- Comprobar los estados azul, verde, amarillo y apagado.
- Después de 10 s, verificar un nuevo arranque con CAUSA=temporizador RTC.

```
BOOT=1
CAUSA=encendido o reinicio, no deep sleep
ESTADO=ACTIVO | LED verde | Tarea de 5 segundos.
ESTADO=SLEEP | Entrando en deep sleep.
BOOT=2
CAUSA=temporizador RTC
CONTADORES timer=1, ext0=0
```

Prueba B - Despertar externo EXT0

- Reiniciar la simulación y esperar el mensaje ESTADO=SLEEP.
- Pulsar el botón azul WAKE EXT0 o mantener la tecla B.
- Verificar un nuevo arranque con CAUSA=pin externo EXT0 (GPIO33).

```
ESTADO=SLEEP | Entrando en deep sleep.
BOOT=2
CAUSA=pin externo EXT0 (GPIO33)
CONTADORES timer=0, ext0=1
```

Automatización disponible

Los archivos `tests/timer-wakeup.yaml` y `tests/ext0-wakeup.yaml` reproducen estas secuencias con Wokwi CLI. La ejecución remota por CLI requiere un token personal `WOKWI_CLI_TOKEN`; los escenarios se entregan listos, pero no se incluyó una credencial privada en el repositorio.

ALCANCE

La captura demuestra el circuito publicado. Las compilaciones y validaciones fueron ejecutadas. Los bloques seriales anteriores definen la salida esperada por los escenarios; no se presentan como una medición física.

7. Análisis del comportamiento

Antes y después del reposo

Antes de dormir se apaga el LED, se vacía el buffer UART y se habilitan las fuentes TIMER y EXT0. Durante deep sleep la CPU y los periféricos digitales se apagan. Al despertar, el chip reinicia el firmware desde setup(), pero conserva las variables ubicadas en memoria RTC.

Impacto energético

El ahorro proviene de mantener el sistema activo únicamente durante la tarea y la preparación, y usar deep sleep durante la espera. El ciclo de demostración usa 10 s de reposo para que la prueba sea rápida. En una aplicación alimentada por batería conviene aumentar ese intervalo de acuerdo con la frecuencia real de muestreo o reporte.

Simulación frente a hardware real

Aspecto	Wokwi	Hardware real
Flujo y temporización	Valida estados y mensajes.	Confirma tiempos con tolerancias reales.
Cableado lógico	Detecta conexiones y pines.	Puede revelar ruido, rebote y falsos contactos.
Causa de despertar	Permite TIMER y EXT0.	Verifica nivel eléctrico y comportamiento RTC.
Consumo	No es una medición energética.	Debe medirse con amperímetro o analizador.
Placa DevKit	Modelo funcional idealizado.	Regulador, USB-UART y LED elevan el consumo.

HARDWARE REAL

No se fabricó ni midió un montaje físico en esta entrega. Por honestidad técnica, no se atribuyen valores de corriente a la simulación. La evidencia física opcional debe añadir fotografías, modelo de instrumento, tensión, corriente activa y corriente en reposo.

Criterios para una medición posterior

- Alimentar a tensión conocida y medir corriente en serie con resolución de microamperios.
- Separar el consumo del módulo ESP32 del regulador, convertidor USB-UART y LED de la placa.
- Repetir las mediciones en estado activo y deep sleep, documentando promedio y picos.

8. Conclusiones y recomendaciones

Conclusiones

- 1. Deep sleep reduce la actividad del sistema de forma drástica y resulta adecuado cuando la aplicación puede reiniciar su flujo después de cada despertar.
- 2. Combinar TIMER y EXT0 permite atender tanto tareas periódicas como eventos externos; el evento que ocurre primero determina la causa del nuevo arranque.
- 3. La memoria RTC y los mensajes seriales estructurados facilitan demostrar que el sistema durmió, despertó y conservó información mínima entre ciclos.

Recomendaciones

- 1. Dimensionar el intervalo de reposo según la necesidad real de la aplicación y mantener apagadas las cargas externas que no sean necesarias.
- 2. Validar siempre el consumo en hardware real, incluyendo regulador, interfaz USB y resistencias de polarización, porque la simulación no representa todas las pérdidas de la placa.

Entregables

Elemento	Ubicación / enlace
Código y README	Repositorio GitHub público
Circuito Wokwi	Proyecto público 471748304593999873
Firmware PlatformIO	.pio/build/esp32dev (generado localmente)
Informe	output/pdf/Informe_Tarea_6_Gestion_Energia_ESP32.pdf
Video YouTube	No incluido; requiere grabación y narración del estudiante

Referencias

- [1] Espressif Systems. [ESP-IDF: Sleep Modes - ESP32](#). Consulta: 07-08-2026.
- [2] Espressif Systems. [Arduino ESP32: Deep Sleep](#). Consulta: 07-08-2026.
- [3] Wokwi. [ESP32 Simulation](#). Consulta: 07-08-2026.
- [4] Wokwi. [Configuring Your Project \(wokwi.toml\)](#). Consulta: 07-08-2026.